3(a): Compare CUBIC, Reno, and BBR

We compared CUBIC, Reno, and BBR at three destinations, using three 60-second runs per algorithm and one-second sampling: 27 matched runs. Each algorithm was set on the data socket before connecting, read back, and recorded. The host ran Linux 6.18.33.2-microsoft-standard-WSL2 and Python 3.14.8. Its recorded library versions were matplotlib 3.11.2 and pydantic 2.13.5.

Destinations were shuffled with seed 422. Algorithm order rotated across repetitions, placing each algorithm first, second, and third once per destination. Dallas used BBR/CUBIC/Reno, CUBIC/Reno/BBR, then Reno/BBR/CUBIC; Netfiber and the IP destination began with the second and third orders, respectively. Tests ran sequentially with a two-second pause. Dallas was the first complete group and became the representative without selecting by throughput. Five incomplete destination groups were excluded; their partial measurements and all failures remain in the logs.

Goodput uses changes in acknowledged bytes divided by actual elapsed time. Algorithm means average three duration-weighted run means; ± values are sample standard deviations across runs. RTT is the mean sampled smoothed RTT. Retransmission totals are a loss proxy, not packet-loss probabilities. The CSV summaries additionally provide per-run minimum, median, arithmetic mean, and linearly interpolated P95 goodput.

Dest.	Algorithm	Goodput (Mbit/s)	RTT (ms)	Retrans.
D1	cubic	273.96 ± 5.63	68.73	224
D1	reno	281.30 ± 2.76	73.65	444
D1	bbr	269.79 ± 11.69	64.53	196
D2	cubic	118.08 ± 4.16	181.90	23
D2	reno	119.07 ± 0.89	181.67	41
D2	bbr	106.59 ± 17.37	177.39	398
D3	cubic	163.12 ± 17.51	132.58	120
D3	reno	163.52 ± 11.80	136.69	11
D3	bbr	147.47 ± 6.02	150.60	541

D1: dal.speedtest.clouvider.net:5200
D2: speed.netfiber.net.il:5201
D3: 96.45.41.167:30001

Reproduce: python main.py analyze --manifest results/q3-test/manifest.json

Raw data and full min/median/mean/P95 tables: results/q3-test/

Repository
Data socket selection
Experiment runner
Statistics and plots

3(a): Observations and limitations

Dallas: Reno delivered 281.30 Mbit/s, CUBIC 273.96, and BBR 269.79. BBR had the lowest mean RTT (64.53 ms), while Reno had the highest (73.65 ms). Thus Reno's modest throughput advantage came with higher measured delay. Reno's median cwnd rose from 1,982 to 7,854 segments across repetitions while throughput stayed near 279–284 Mbit/s. A larger window budget did not yield proportionally more delivery. Minimum filtered RTT was about 29–32 ms; higher observed RTT is consistent with queueing or other variable delay, including reverse-path delay.

Netfiber: With minimum RTT around 170–174 ms, Reno and CUBIC achieved similar throughput (119.07 versus 118.08 Mbit/s). Their difference was small relative to CUBIC's run variation. BBR averaged 106.59 Mbit/s and slightly lower RTT. Its third run delivered only 86.57 Mbit/s with 396 retransmissions, compared with 115.56 and 117.63 Mbit/s in its first two runs. RTT stayed close to baseline during much of the slowdown, showing why a loss signal is useful alongside RTT.

IP destination: Reno and CUBIC were effectively tied (163.52 versus 163.12 Mbit/s), with run standard deviations of 11.80 and 17.51 Mbit/s. BBR averaged 147.47 Mbit/s and had more variable RTT. In its first run, the interval at about 50 seconds had zero acknowledged goodput and a 1,559.64 ms RTT, with retransmissions increasing. The transfer still completed. This stall contributes to the averages; it was not removed as an outlier.

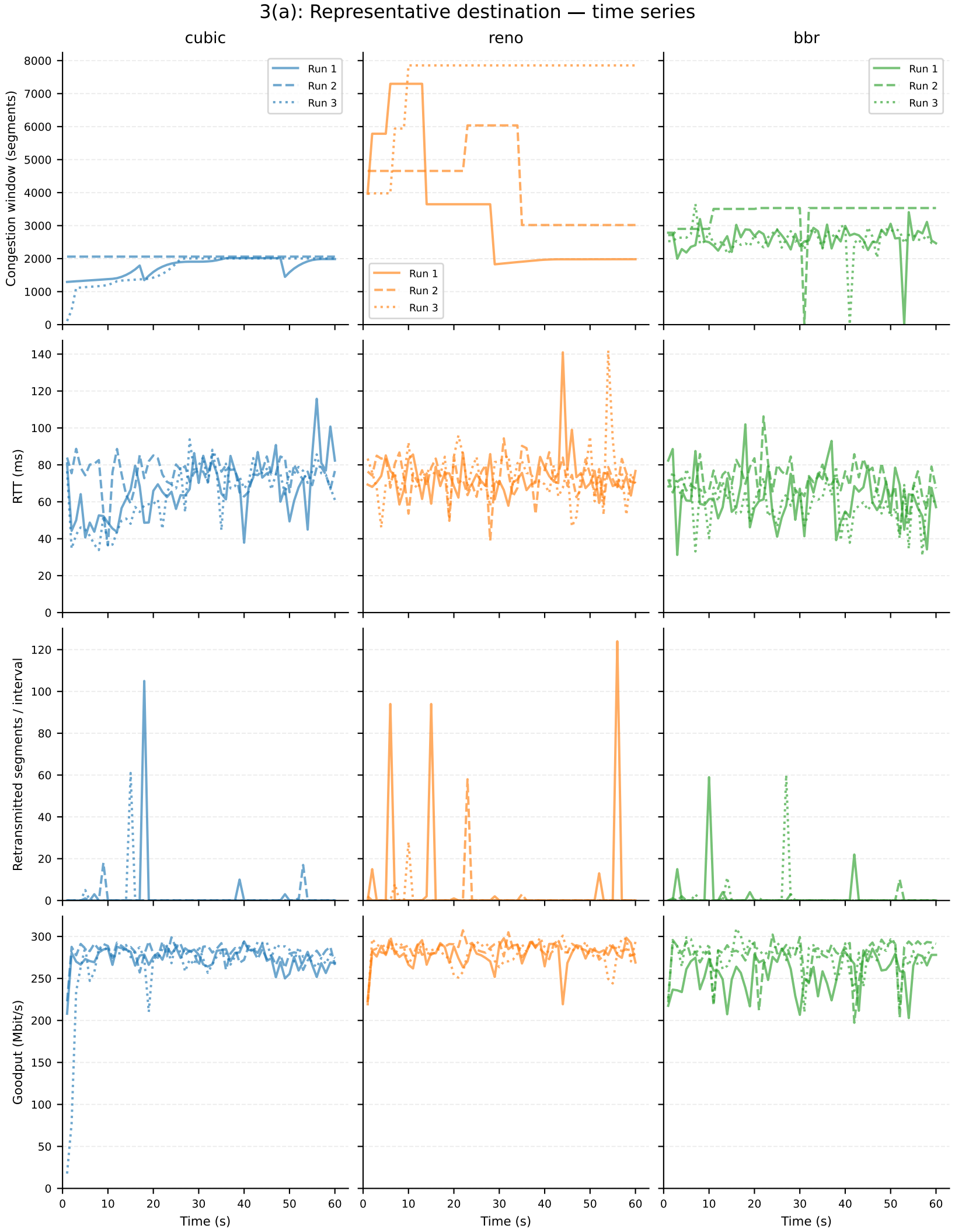
Across these paths, Reno had the highest mean throughput, BBR had lower mean RTT on two paths, and neither CUBIC nor BBR established a throughput win. These are observed operating regimes, not universal rankings. Reno's additive growth [1], CUBIC's faster growth on suitable large-bandwidth-delay paths [2], and BBR's bandwidth/RTT model [3] explain their intended behavior, but do not prove what caused differences between sequential Internet tests.

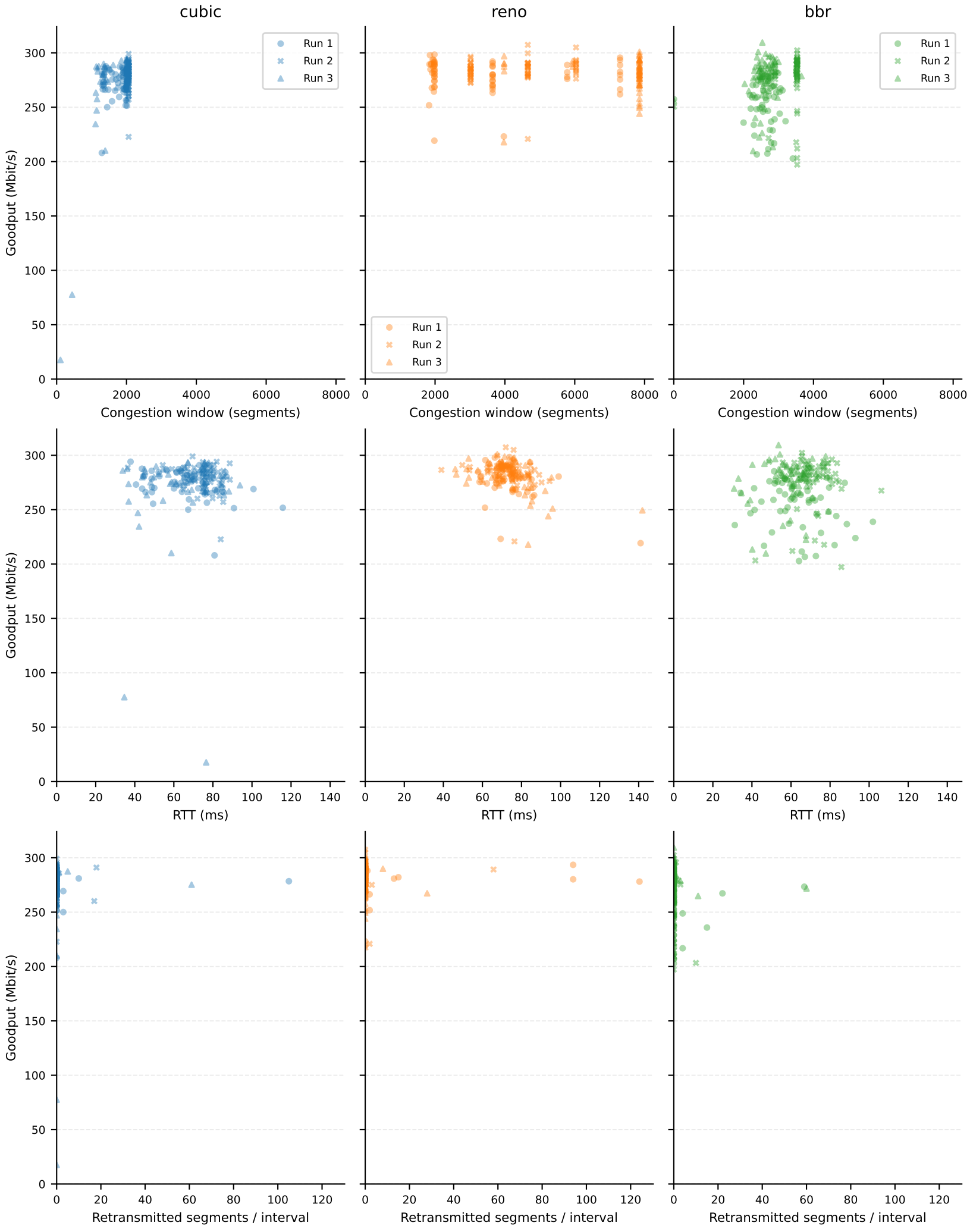
For a window-limited flow, rate is approximately cwnd × MSS / RTT. In Dallas's third Reno run, median window budget was about 11.37 MB, versus a 1.09 MB achieved-rate × minimum-RTT proxy. The proxy is not an independent bottleneck-capacity estimate, and cwnd is not actual bytes in flight. Server limits, receiver windows, pacing, or path capacity can limit delivery. Public routes and cross-traffic changed between tests; rotating order cannot remove that variation. Three runs and one-second samples also limit conclusions. These single-flow tests do not establish fairness.

[1] RFC 5681: Reno congestion avoidance

[2] RFC 9438: CUBIC

[3] Google BBR documentation


3(a): Representative destination — time series
cubic
reno
bbr
Run 1
Run 2
Run 3
Congestion window (segments)
RTT (ms)
Retransmitted segments / interval
Goodput (Mbit/s)
Time (s)


3(a): Representative destination — scatter plots
cubic
reno
bbr
Run 1
Run 2
Run 3
Goodput (Mbit/s)
Congestion window (segments)
RTT (ms)
Retransmitted segments / interval

3(b): Proposed congestion control algorithm

The proposed controller uses smoothed RTT, acknowledged goodput, and new loss-recovery events. Start with cwnd = 10 segments and ssthresh = infinity; minimum cwnd is 2. Update once per RTT after ACK progress. Let base be the minimum valid RTT over 10 seconds, q = max(0, srtt/base − 1), and B be the maximum non-application-limited epoch goodput over 10 seconds. Use seconds and bytes/s for BDP = max(2, B × base / MSS). Smooth goodput with rate = 0.75 × old_rate + 0.25 × epoch_goodput. Initialize rate from the first valid epoch.

```
State: cwnd = 10; mode = STARTUP; probe_age = 0
On entering a new recovery episode, once per episode:
    cwnd = max(2, floor(0.70 * cwnd)); ssthresh = cwnd
    mode = AVOIDANCE; reset epoch and probe accounting
On retransmission timeout, through the TCP stack:
    ssthresh = max(2, floor(cwnd / 2)); cwnd = 2
    mode = AVOIDANCE; reset epoch and probe accounting
Once per RTT with new ACKs, outside recovery:
    update base, BDP, q, rate; increment probe_age
    if app limited or receiver-window limited: hold; finish
    if mode == STARTUP and q < 0.10:
        cwnd += min(newly_acked_segments, cwnd); finish
    mode = AVOIDANCE
    if q > 0.25:
        cwnd = max(2, floor(0.90 * cwnd)); probe_age = 0
    else if q < 0.10:
        if cwnd < 1.25 * BDP or rate > 1.02 * old_rate:
            cwnd += 1
        else if probe_age >= 8:
            cwnd += 1; probe_age = 0
    otherwise hold; enforce kernel cwnd clamp
```

The Dallas window/rate mismatch motivates the throughput gate; Netfiber motivates a separate loss response; the IP-path stall motivates responding to delay inflation. The hold band and smoothing aim to reduce oscillation, and periodic probes avoid remaining at an underestimated rate. The thresholds are hypotheses, not tuned or validated results. The full specification in Q3_DESIGN.md defines recovery boundaries, fractional ACK accounting, and missing/limited-signal handling.

Assuming persistent flows, equal RTTs, and a shared bottleneck, additive growth and proportional reductions may encourage similar windows. Fairness and convergence are not guaranteed: unequal RTTs favor different update rates, delay-sensitive flows can yield to loss-based flows, and stale RTT/rate estimates can preserve unfair allocations. Probing may produce a limit cycle rather than a fixed rate. Assignment 3 must implement and evaluate the proposal; these baseline measurements do not evaluate it.

AI helped with code, plots, checking, and drafting. A teammate collected the Linux/WSL2 measurements.